PROJECT REPORT

ON

BUDGET MANAGEMENT SYSTEM

A Python Based Desktop Application using CustomTkinter and SQLite

Submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF SCIENCE (B.Sc.)

Submitted By

SUPANDEEP KAUR

Roll No.: 24053434

Submitted To

Department of Computer Science

GKSM College, Tanda

Affiliated to Panjab University, Chandigarh

Academic Session: 2026-2027

DECLARATION

I hereby declare that the project work entitled “Budget Management System” is my own work carried out as part of my Bachelor of Science (B.Sc.) academic/internship project.

The project has been developed using Python programming language with CustomTkinter for the graphical user interface and SQLite for storing application data. The work presented in this report describes the analysis, design, development and testing of the proposed system.

I further declare that this project report has not been submitted earlier for the award of any other degree or qualification to any institution or university.

Name	Roll No.	Signature
Supandeep Kaur	_____	_____

Date: _____

CERTIFICATE / TO WHOM IT MAY CONCERN

This is to certify that the project report entitled “Budget Management System” submitted by Supandeep Kaur, in partial fulfillment of the requirements for the degree of Bachelor of Science (B.Sc.), is a record of project work carried out by the student during the academic/ internship period.

The project focuses on the development of a simple desktop application for recording and managing personal budget information. The project uses Python, CustomTkinter and SQLite and includes user account handling, budget entry, spending records, balance information and graphical representation.

To the best of our knowledge, the project work is suitable for academic evaluation and demonstrates the student’s practical understanding of programming, GUI development and database connectivity.

Project Guide	**Project Supervisor**
Miss. Nishtha	Prof. Rajandeep Kaur
Signature: _____	Signature: _____

Date: _____

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to everyone who supported and guided me during the development of my Budget Management System project.

First of all, I am thankful to my college, GKSM College, Tanda, for providing the academic environment and facilities required to complete this project. I am also thankful to the Department of Computer Science for providing guidance related to programming, database concepts and project documentation.

I would also like to thank my project/internship mentors and the team at O7 Services for providing practical exposure to Python programming and helping me understand how a small software project is planned and developed.

Finally, I am grateful to my teachers, friends and family members for their encouragement and support throughout the project.

Supandeep Kaur

PREFACE

This project report has been prepared as documentation of the development of a Budget Management System. The main purpose of the project is to provide a simple computer-based way to enter, store, update and review budget-related information.

The application is developed as a beginner-friendly desktop application. The graphical interface is created using CustomTkinter, while SQLite is used as the local database. The application contains pages for login, account creation, dashboard and budget-related information. Additional sections are provided for spending, budget, balance and expense graph functionality.

The report describes the complete project process, beginning with problem definition and objectives, followed by methodology, analysis, design, implementation, testing, evaluation and conclusion.

The report has been arranged in chapters so that the development of the system can be understood step by step.

TABLE OF CONTENTS

Section	Page
Project Report Cover	1
Declaration	2
Certificate	3
Acknowledgement	4
Preface	5
Chapter 1 – Introduction	7
1.1 Problem Definition	7
1.2 Objectives and Scope	8
1.3 Minimum Requirements	9
1.4 Deliverables	9
1.5 Project Schedule	10
1.6 Relevance to Degree Programme	10
Chapter 2 – Project Methodology	11
2.1 Methodology Comparisons	11
2.2 SDLC	12
2.3 Iterative/Prototyping Approach	13
2.4 Chosen Methodology	14

Section	Page
Chapter 3 – Project Analysis	15
3.1 Introduction / Intended User	15
3.2 Existing Practices	16
3.3 Feasibility Study	17
3.4 User Requirements	19
3.5 Hardware and Software Requirements	20
Chapter 4 – Project Design	21
4.1 Introduction	21
4.2 System Architecture	22
4.3 Flowchart	23
4.4 Data Flow	24
4.5 Database Design	25

4.6 Decision Table 26

5.3 GUI Development 29

5.4 Database Connectivity 30

5.5 Modules 31

5.6 Screenshots 32

Section	Page
Chapter 6 – Project Testing / Results	33
6.1 Test Plan	33
6.2 Test Cases	34
6.3 Unit Testing	35
6.4 User Testing	35
6.5 Interface Testing	36
6.6 Verification and Validation	36
6.7 Setup and Maintenance	37
Chapter 7 – Implementation Evaluation	38
7.1 Effectiveness of Testing	38
7.2 Limitations	38
7.3 Future Scope	39
Chapter 8 – Conclusion	40
Bibliography / References	40

CHAPTER 1 – INTRODUCTION

1.1 Problem Definition

Managing personal income and expenses manually can become difficult when the user has several expenses during a week or month. Information written in notebooks or kept in separate notes may be difficult to organize and review.

The Budget Management System is designed to provide a simple computerized solution for recording budget information. The user can create an account, log in and enter information such as date, income and spending categories. The dashboard acts as the main working area of the application.

The system is not designed as a banking application. It is a personal record and budget planning tool. Therefore, the focus is on storing and reviewing records rather than providing banking operations such as withdrawals.

The project demonstrates how Python GUI programming can be combined with a local database to create a practical desktop application.

1.2 PROJECT OBJECTIVES AND SCOPE

The main objective of the project is to develop an easy-to-use desktop Budget Management System for maintaining personal financial records.

Provide a login system for registered users.
Allow a new user to create an account.
Provide a central dashboard for budget information.
Allow the user to enter income and expense information.
Organize expenses into categories such as Food, Transport, Shopping, Bills, Education, Health and Entertainment.
Provide spending, budget and balance views.
Provide graphical representation of expense information.
Store account and budget information in an SQLite database.

Provide simple messages when required fields are missing or passwords do not match.

Keep the interface simple enough for a beginner user.

Scope: The project is intended for personal budget record management on a computer. It can be extended later with reports, export facilities, improved charts and additional account controls.

1.3 MINIMUM REQUIREMENTS

The system requires a basic computer capable of running Python and a desktop graphical interface.

Category	Minimum Requirement
Processor	Intel Core i3 or equivalent
RAM	4 GB or above
Storage	At least 500 MB free for project files and database
Operating System	Windows 10/11 or compatible desktop OS
Python	Python 3.x
GUI Library	CustomTkinter
Database	SQLite
IDE	Visual Studio Code or similar Python IDE
Database Viewer	DB Browser for SQLite (optional)

The application uses a local SQLite database, so a separate database server is not required.

1.4 DELIVERABLES

Complete Python source code for the Budget Management System.

SQLite database containing the required application tables.

Login and account creation interface.

Dashboard for entering budget information.

Spending, budget and balance sections.

Expense graph section.

Testing records and results.

Project report and presentation material.

1.5 PROJECT SCHEDULE

Activity	Expected Work
Requirement Analysis	Identify pages, fields and functions
Database Design	Create SQLite tables and relationships
Interface Design	Design CustomTkinter pages
Implementation	Develop login, signup and dashboard
Database Integration	Connect forms to SQLite
Testing	Test valid and invalid inputs
Documentation	Prepare report and presentation

1.6 RELEVANCE TO DEGREE PROGRAMME

The project is directly related to computer science because it combines programming, graphical user interface development, database management and software development

practices.

- Python programming is used to implement application logic.
- CustomTkinter is used for GUI design and event handling.
- SQLite provides practical database storage.
- SQL statements are used to insert and retrieve records.
- Conditional statements and validation are used to handle user input.
- Functions and class methods are used to organize the program.
- Software development methodology is used to plan and test the system.

The project also provides practical experience in converting a real-world requirement into a working computer application.

CHAPTER 2 – PROJECT METHODOLOGY

2.1 METHODOLOGY COMPARISONS

Project management methodologies provide an organized way to plan, develop and test software. The sample report discusses SDLC-related approaches and compares development methods before selecting a suitable approach.

Method	Main Idea	Suitability for This Project
Waterfall	Sequential phases	Less flexible for changing GUI requirements
Prototyping	Build and improve an early model	Very suitable for GUI projects
Iterative	Develop and improve in repeated cycles	Suitable
RAD	Fast development using prototypes/tools	Suitable for small projects
Agile	Frequent changes and feedback	Useful but more formal than needed

For this beginner desktop application, an iterative/prototyping approach is appropriate because pages can be created, tested and modified one at a time.

2.2 INFORMATION SYSTEMS DEVELOPMENT LIFE CYCLE

The System Development Life Cycle (SDLC) provides a structured process for developing an information system. The major activities are requirements, design, implementation, testing, deployment and maintenance.

Requirement Analysis – identify what the application must do.

Design – plan screens, database tables, navigation and data flow.

Implementation – write Python and SQL code.

Testing – check functions, inputs, database operations and navigation.

Deployment – run the completed application in the intended environment.

Maintenance – correct errors and improve the application when requirements change.

The project follows these stages in a simplified form suitable for a small academic application.

2.3 ITERATIVE / PROTOTYPING APPROACH

In an iterative approach, the software is developed in smaller parts. Each part is tested and improved before the next part is finalized.

1. Create the basic project window and theme.
2. Create the login page.
3. Create the account registration page.
4. Connect account registration with the SQLite database.
5. Create the dashboard and budget input fields.
6. Add navigation to spending, budget, balance and graph pages.
7. Connect budget information with database operations.
8. Test validation messages and database operations.
9. Improve layout, labels and navigation based on testing.

This approach is especially useful for the project because the GUI can be visually checked after each page is developed.

2.4 CHOSEN METHODOLOGY

The chosen methodology for the project is an iterative development approach supported by prototyping. The system was not treated as a single large program from the beginning. Instead, individual pages and functions were developed and connected progressively.

Phase	Project Activity	Output
Requirements	Identify pages and operations	Requirement list
Design	Plan GUI and database	Page/database design
Prototype	Create initial GUI	Working interface
Implementation	Add Python and SQL logic	Functional modules
Testing	Check each function	Test results
Review	Correct errors and improve layout	Improved system
Finalization	Prepare documentation	Project report

The methodology keeps the development process understandable and makes it easier to locate errors.

CHAPTER 3 – PROJECT ANALYSIS

3.1 INTRODUCTION / INTENDED USER

Analysis is the stage in which the existing problem is studied and the requirements of the proposed system are identified.

The intended user of the Budget Management System is a person who wants to maintain a simple personal record of income, expenses, budget and balance information on a computer.

The system is designed with a simple interface so that the user does not need advanced computer knowledge. The main operations are performed through buttons and entry fields.

The project also contains account-related functionality so that the budget information can be associated with a registered username.

3.2 EXISTING USER PRACTICES

Before using a computerized budget system, a user may record financial information in notebooks, mobile notes or separate spreadsheets. These methods can work for small amounts of information but may become inconvenient as records increase.

Existing Practice	Problem
Notebook	Searching old records can take time
Mobile Notes	Records may not be structured
Separate spreadsheets	Requires spreadsheet knowledge
Mental calculation	Can lead to missed or incorrect totals
Multiple files	Information becomes difficult to organize

The proposed system brings important information into one application. It does not attempt to replace professional accounting software; it provides a simple personal record tool.

3.3 FEASIBILITY STUDY

Economic Feasibility

The project is economically feasible because it uses Python, CustomTkinter and SQLite, which can be used without purchasing an expensive database server or specialized commercial development software.

Technical Feasibility

The system is technically feasible because Python provides the programming environment, CustomTkinter provides GUI components and SQLite provides local database storage. The application can run on a normal computer with Python installed.

Operational Feasibility

The application is operationally feasible because the intended operations are simple: login, account creation, entering budget data, viewing records and checking graphical information.

Schedule Feasibility

The project is suitable for an academic/internship schedule because it can be divided into small modules and tested progressively.

3.4 USER REQUIREMENTS

Functional Requirements

The user shall be able to create an account.
The user shall be able to log in using registered credentials.
The system shall validate required fields.
The system shall detect a password and confirmation mismatch.
The system shall prevent duplicate usernames.
The user shall be able to enter budget information.
The user shall be able to manage spending/budget records.
The system shall display balance-related information.

The system shall provide an expense graph.

The database shall store relevant records.

Non-Functional Requirements

Simple and understandable interface.

Reasonable response time for local database operations.

Clear error and success messages.

Maintainable Python code.

Local database storage.

Consistent navigation between pages.

3.5 SOFTWARE AND HARDWARE REQUIREMENTS

Hardware	**Requirement**
CPU	Intel Core i3 or equivalent
RAM	4 GB minimum
Storage	500 MB or more free
Display	Standard color monitor
Keyboard/Mouse	Standard input devices
Software	**Requirement**
Operating System	Windows 10/11 or compatible
Programming Language	Python 3.x
GUI	CustomTkinter
Database	SQLite
IDE	Visual Studio Code

Database Tool	DB Browser for SQLite

The requirements can be increased if the project is extended with larger datasets or additional reporting features.

CHAPTER 4 – PROJECT DESIGN

4.1 INTRODUCTION

Design is the stage in which the requirements are converted into a structured solution. The sample report divides design into preliminary/general design and structured/detailed design.

The preliminary design of this project defines the pages and navigation. The detailed design defines the input fields, buttons, database operations and page-level functions.

- Welcome/Login page
- Create Account page
- Dashboard page
- Spending page
- Budget page
- Balance page
- Expense Graph page
- Additional project pages such as Savings/Thank You where implemented

The design emphasizes simple navigation and clear separation of user interface and database operations.

4.2 SYSTEM ARCHITECTURE

The application follows a simple three-part structure: user interface, application logic and database.

Layer	Responsibility
User Interface	CustomTkinter windows, frames, entries, labels and buttons
Application Logic	Validation, calculations, navigation and event functions

Database Layer	SQLite connection, SQL INSERT/SELECT/ UPDATE/DELETE operations

The user enters information through the GUI. Python functions process the information and communicate with the SQLite database. Retrieved information can then be displayed on the relevant page.

4.3 FLOWCHART

The overall flow of the application can be represented as follows:

Step	Process
1	Start application
2	Show login / welcome interface
3	User chooses Login or Create Account
4	If creating account, validate fields
5	Store new account in SQLite
6	Login using username and password
7	Open Dashboard
8	Enter or manage budget information
9	View Spending / Budget / Balance / Graph
10	Logout or close application
11	End

4.4 DATA FLOW

The main data flow begins with user input and ends with stored or displayed information.

Input	Processing	Output
Name, email, username, password	Validate and insert into users table	Account created / error message
Username, password	Search user record	Login success / invalid login
Date, income, category, amount	Validate and store budget record	Saved record
Stored expense records	Retrieve and calculate totals	Spending/balance information
Expense records	Group or calculate values	Graphical representation

The system uses parameterized SQL values in the shown account-creation code, for example INSERT INTO users(username, password, full_name, email) VALUES (?, ?, ?, ?).

4.5 DATABASE DESIGN

SQLite is used as the local database. The supplied program clearly shows a users table containing username, password, full_name and email fields. Other project tables should contain the budget/spending records required by the implemented pages.

Table / Entity	Important Fields	Purpose
users	username, password, full_name, email	Stores registered user account information
budget/records	date, income, category, amount, user reference	Stores budget and spending information
optional summary data	period, total, savings	Supports summary views if

		implemented

The exact names of additional tables should match the final source code/database used during submission. This report does not invent table names that are not visible in the supplied program snippet.

4.6 DECISION TABLE

Condition	Action	Result
All signup fields filled	Continue validation	Next check
Any signup field empty	Show warning	Account not created
Password != Confirm Password	Show error	Account not created
Username already exists	Catch IntegrityError	Account not created
All checks successful	INSERT into users	Account created
Valid login credentials	Open dashboard	User logged in
Invalid login credentials	Show error	Stay on login page

Decision tables help identify the expected response of the application for different input conditions.

CHAPTER 5 – PROJECT DEVELOPMENT

5.1 IMPLEMENTATION TOOLS AND ENVIRONMENT

The project is implemented as a desktop application. The following tools form the development environment.

Python – main programming language.
CustomTkinter – modern graphical user interface toolkit based on Tkinter.
SQLite – local relational database.
DB Browser for SQLite – optional tool for viewing and checking the database.
Visual Studio Code – code editing and project execution environment.

The development environment is suitable for an academic project because it is lightweight and allows the interface and database to be developed on a normal personal computer.

5.2 PROGRAMMING LANGUAGE USED – PYTHON

Python is the main programming language used in the project. It is a high-level programming language with simple syntax and a large collection of libraries.

Easy-to-read syntax.
Supports object-oriented and procedural programming.
Provides database connectivity through standard libraries.
Supports GUI development through Tkinter and related packages.
Suitable for rapid development and prototyping.

In the supplied program, Python features include functions, class methods, conditional statements, database connections, exception handling and GUI widgets.

For example, the create_account method reads values using .get(), validates them, creates a database connection, executes an INSERT statement and handles sqlite3.IntegrityError.

5.3 GUI DEVELOPMENT USING CUSTOMTKINTER

CustomTkinter is used to create the graphical user interface. It provides modern versions of common Tkinter widgets and allows themes, colors, sizes and layouts to be customized.

Widget / Feature	Use in Project
CTkFrame	Groups related interface elements
CTkEntry	Accepts name, email, username, password and budget input
CTkButton	Performs actions and navigation
CTkLabel	Displays headings and information
CTkScrollableFrame	Provides a scrollable dashboard area
show_page()	Changes between application pages
make_header()	Creates consistent page headers
make_nav()	Creates navigation buttons

The interface shown in the code uses a light green theme and a clean dashboard layout.

5.4 DATABASE CONNECTIVITY

SQLite is connected through Python database operations. A connection is created when the application needs to read or write information.

The supplied account creation code uses the following general process:

1. Read values from the GUI entry fields.
2. Check whether required information has been entered.
3. Compare password and confirmation password.
4. Call get_connection() to open the database.
5. Create a cursor.

6. Execute an INSERT statement with parameter values.
7. Commit the transaction.
8. Show a success message.
9. Clear the entry fields.
10. Return the user to the login page.
11. Close the connection in the finally block.

Using parameter placeholders in SQL is a good practice because values are passed separately from the SQL command.

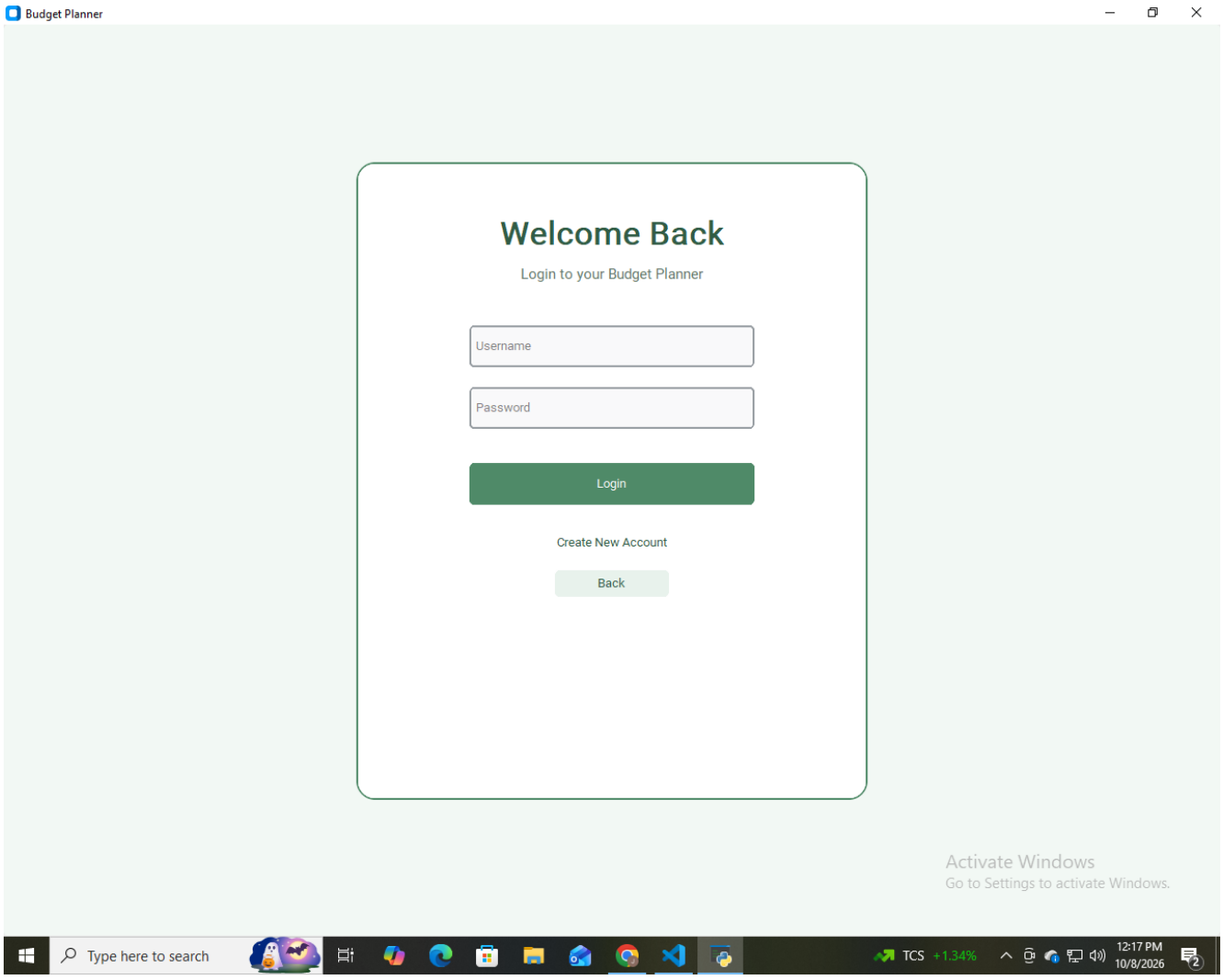
5.5 MAJOR MODULES / PAGES

Module	Main Purpose
Login	Authenticate registered user
Create Account	Register a new user
Dashboard	Enter and manage budget information
Spending	View/manage spending records
Budget	View or manage budget information
Balance	Display balance-related information
Expense Graph	Represent expense information graphically
Database	Store and retrieve application information

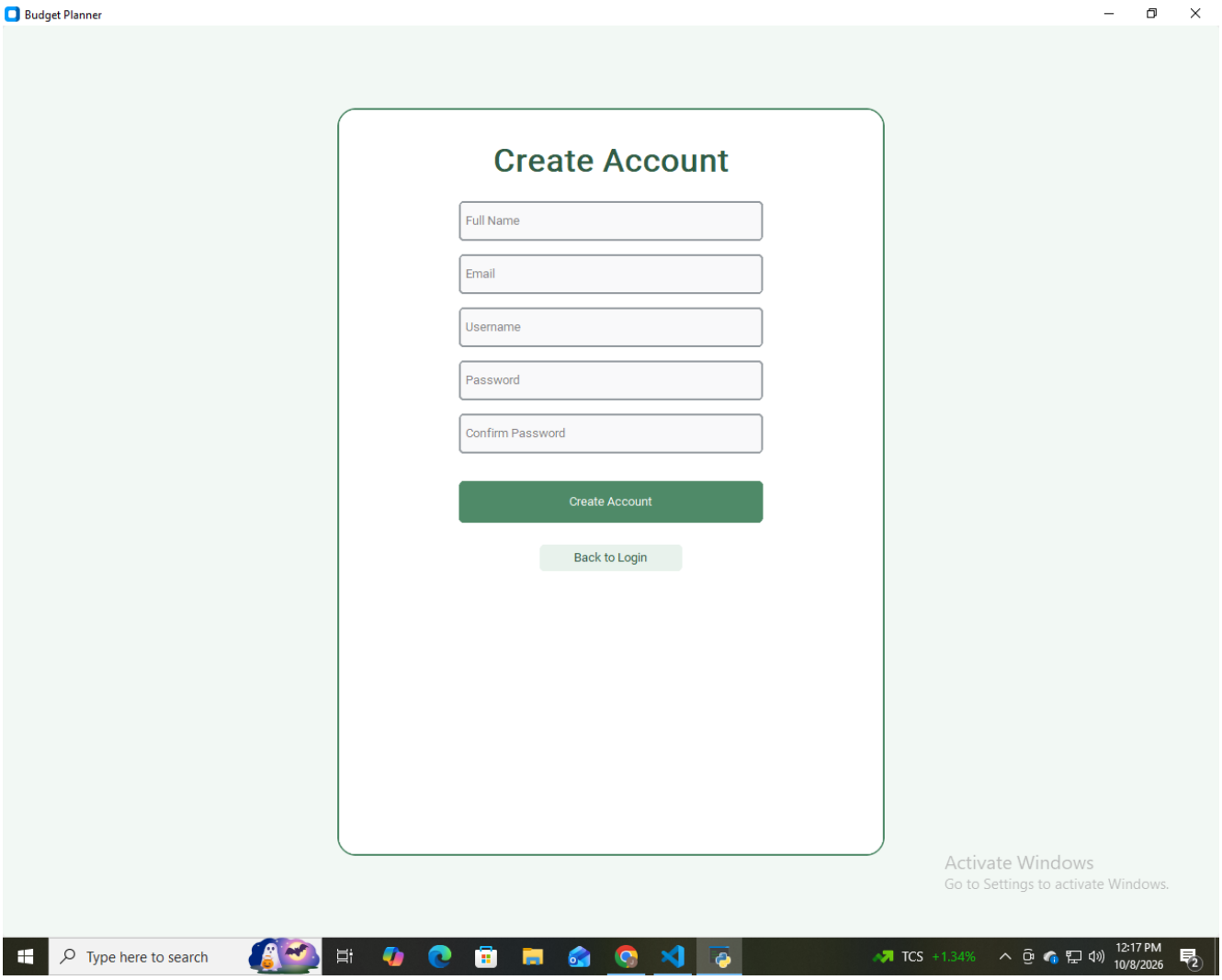
The supplied dashboard includes navigation buttons for Dashboard, Spending, Budget, Balance and Expense Graph. It also creates date and income entry fields and category choices such as Food, Transport, Shopping, Bills, Education, Health and Entertainment.

5.6 SCREENSHOTS / USER INTERFACE

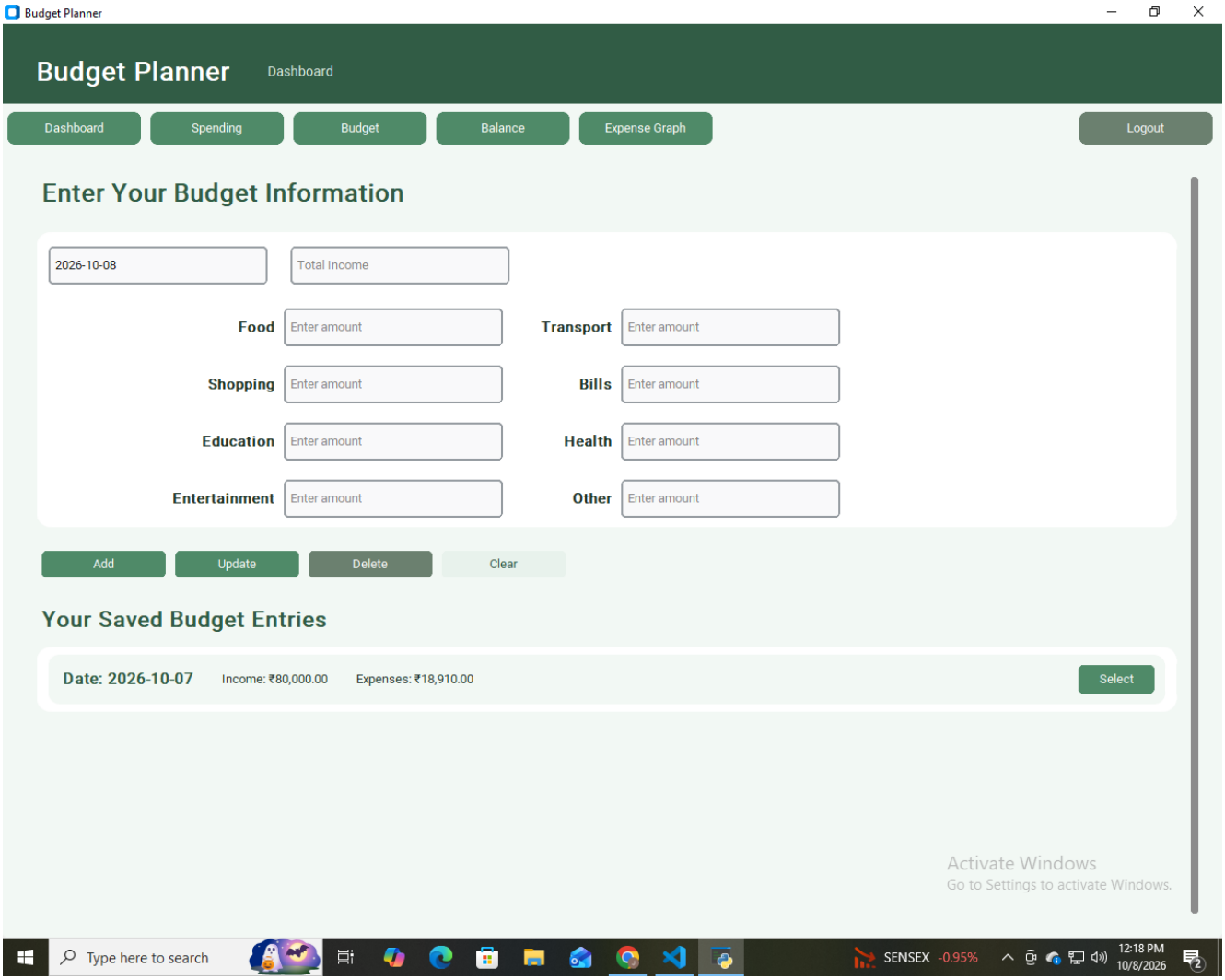
Screenshots are an important part of the project report because they demonstrate the actual interface developed during implementation. The following spaces can be replaced with screenshots from the final running program.



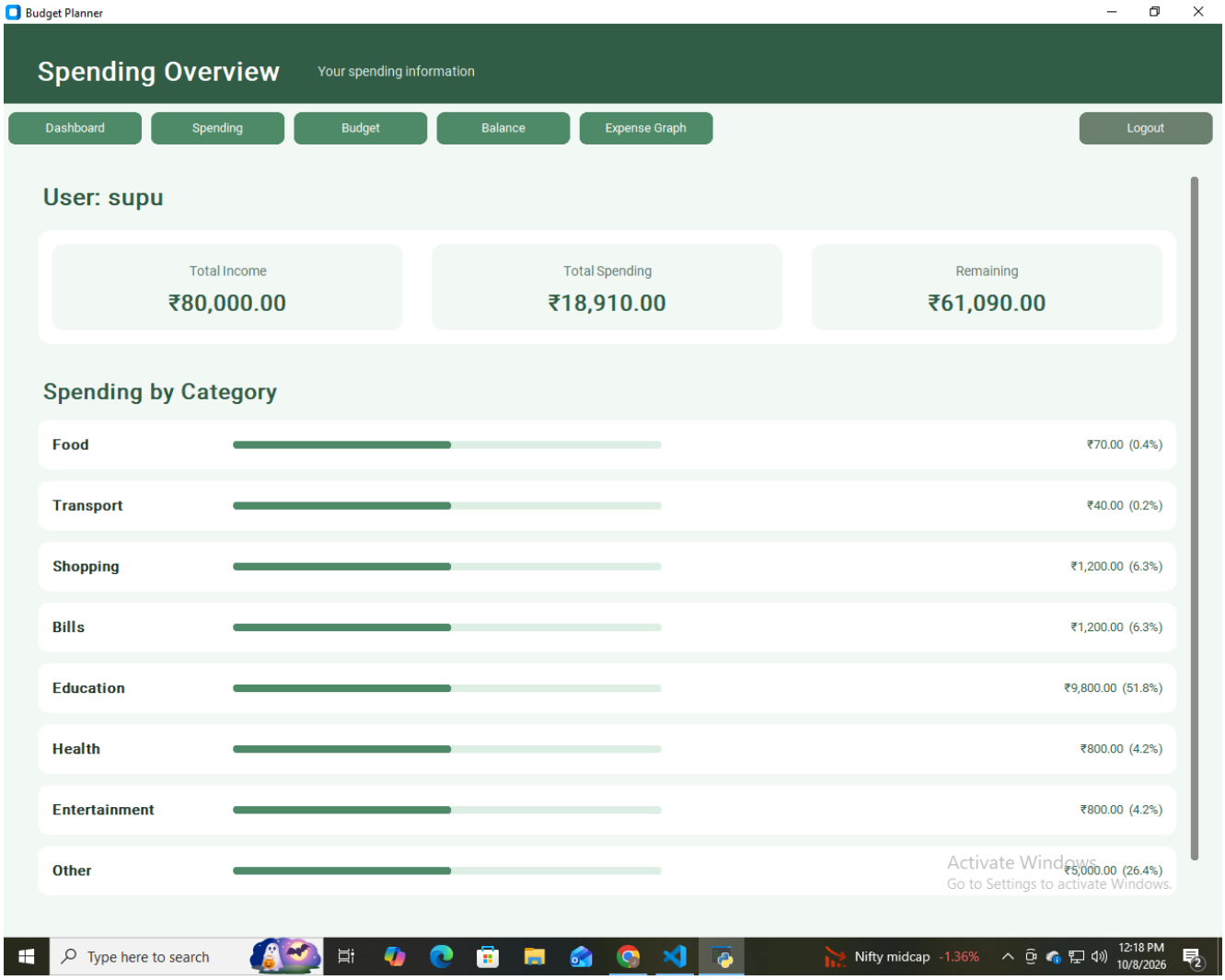
[Login Page]



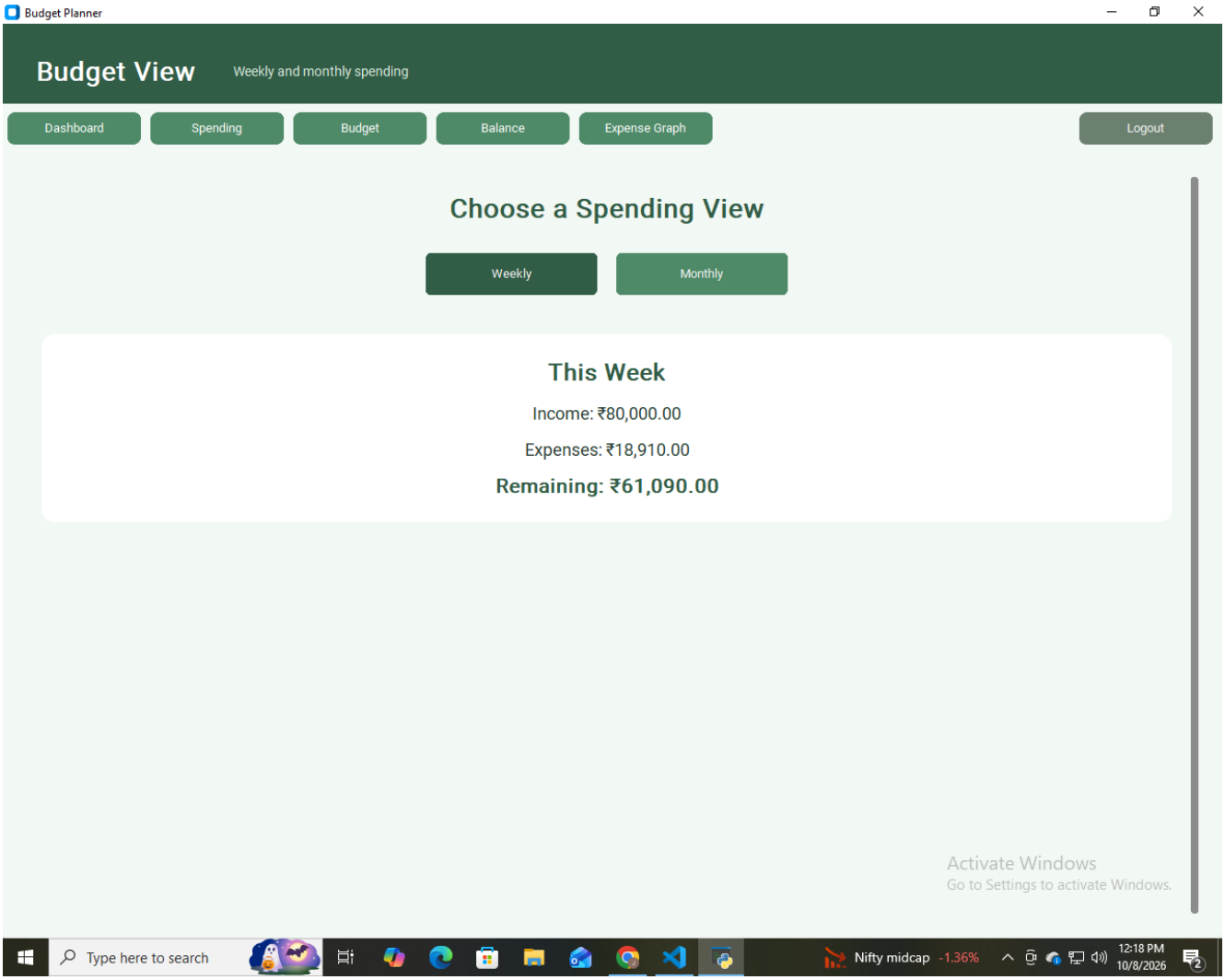
[Create Account]


Budget Planner
Budget Planner
Dashboard
Dashboard
Spending
Budget
Balance
Expense Graph
Logout
Enter Your Budget Information
2026-10-08
Total Income
Food
Enter amount
Transport
Enter amount
Shopping
Enter amount
Bills
Enter amount
Education
Enter amount
Health
Enter amount
Entertainment
Enter amount
Other
Enter amount
Add
Update
Delete
Clear
Your Saved Budget Entries
Date: 2026-10-07
Income: ₹80,000.00
Expenses: ₹18,910.00
Select
Activate Windows
Go to Settings to activate Windows.
Type here to search
SENSEX -0.95%
12:18 PM
10/8/2026

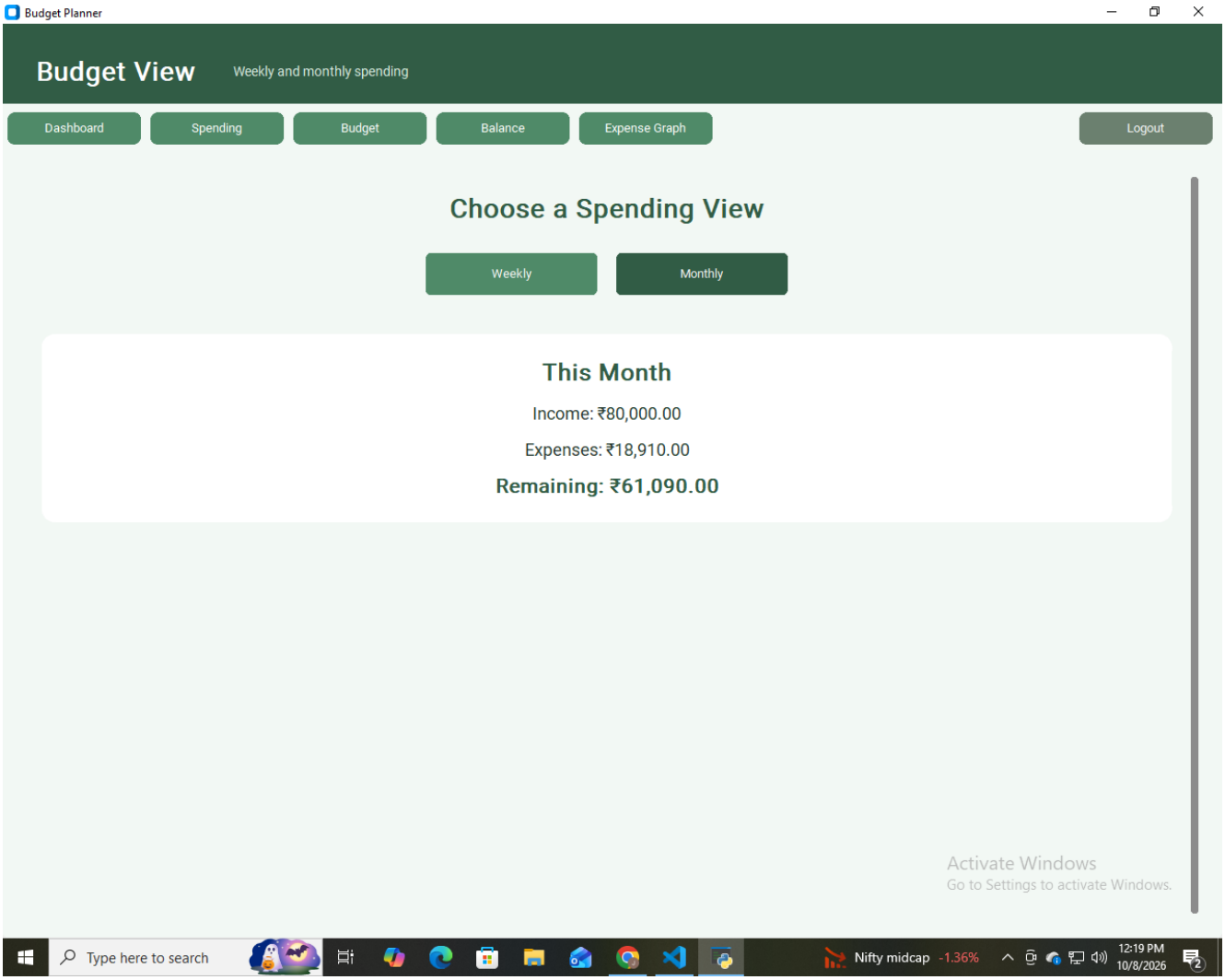
[Dashboard]



[Spending/Budget Page]

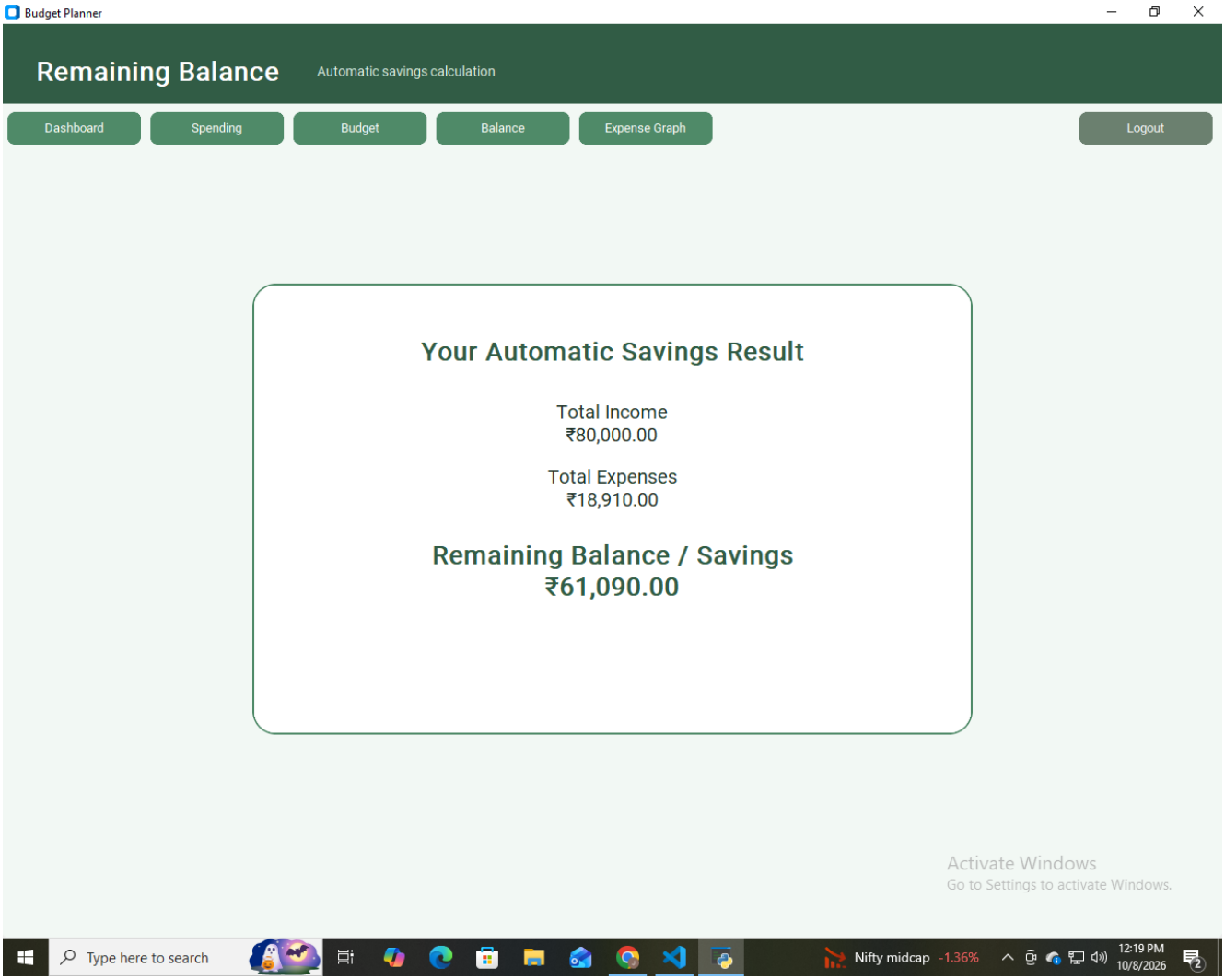


[Weekly Spending]

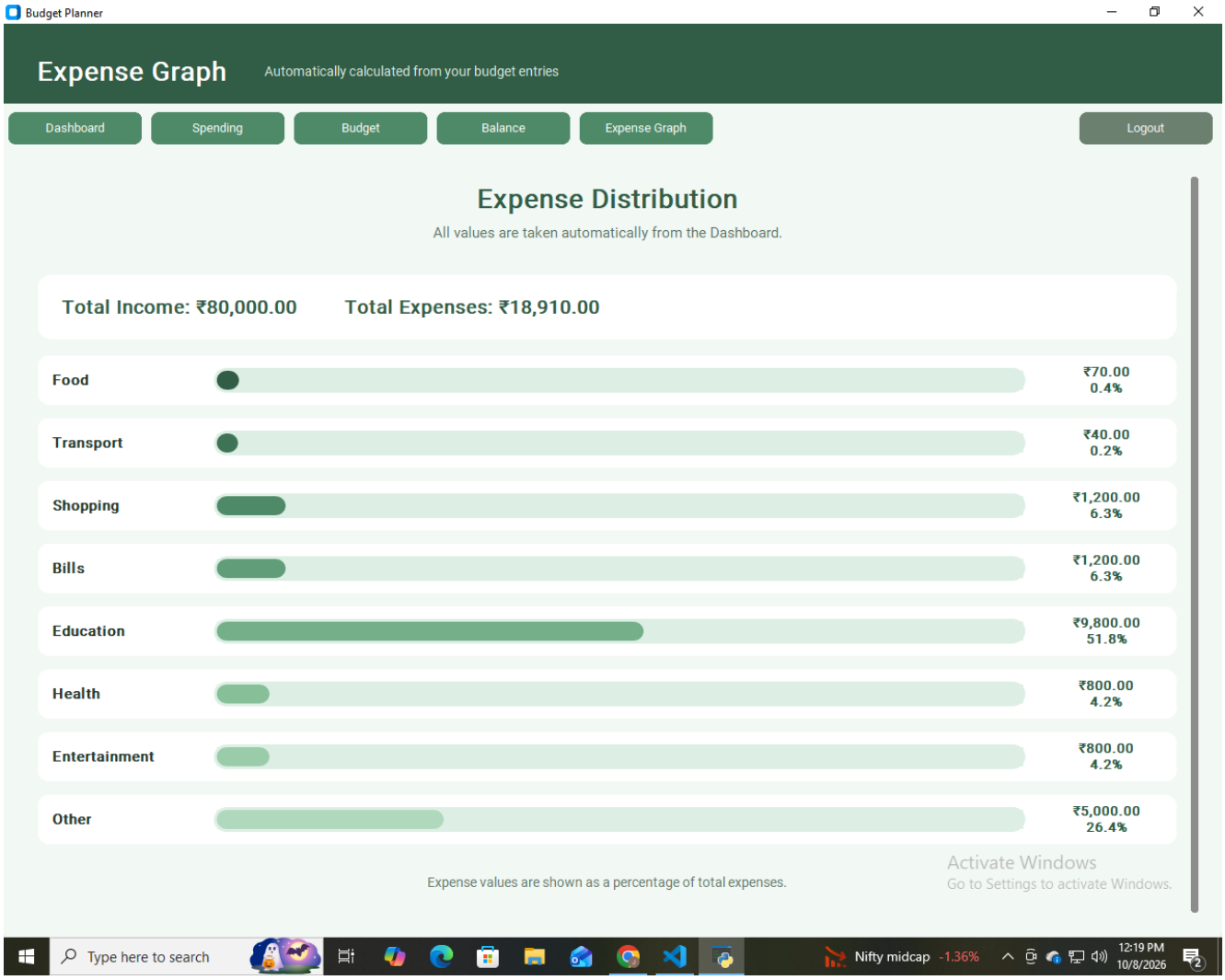


[Monthly Spending]

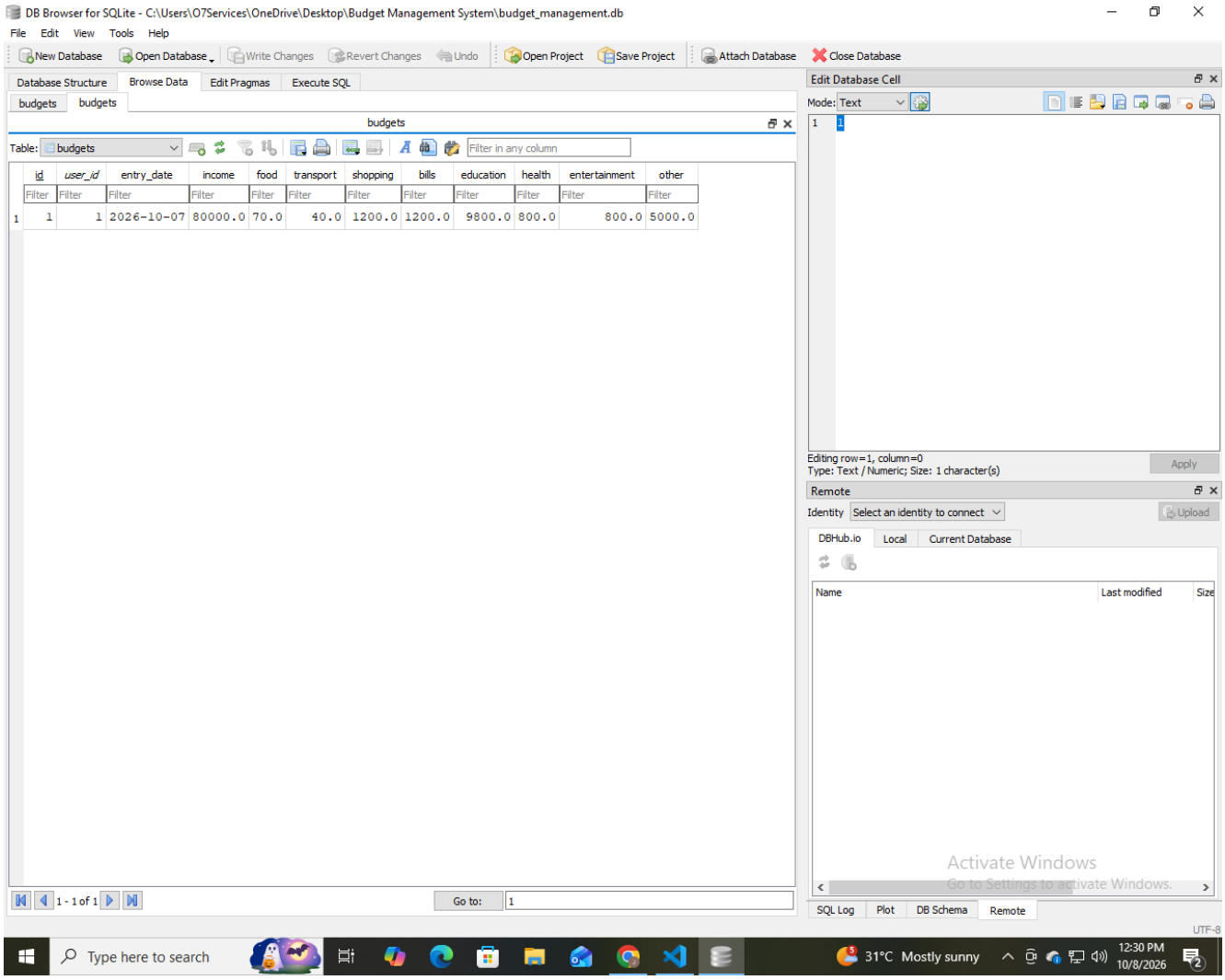
5.6 SCREENSHOTS – CONTINUED



[Balance Page]



[Expense Graph]

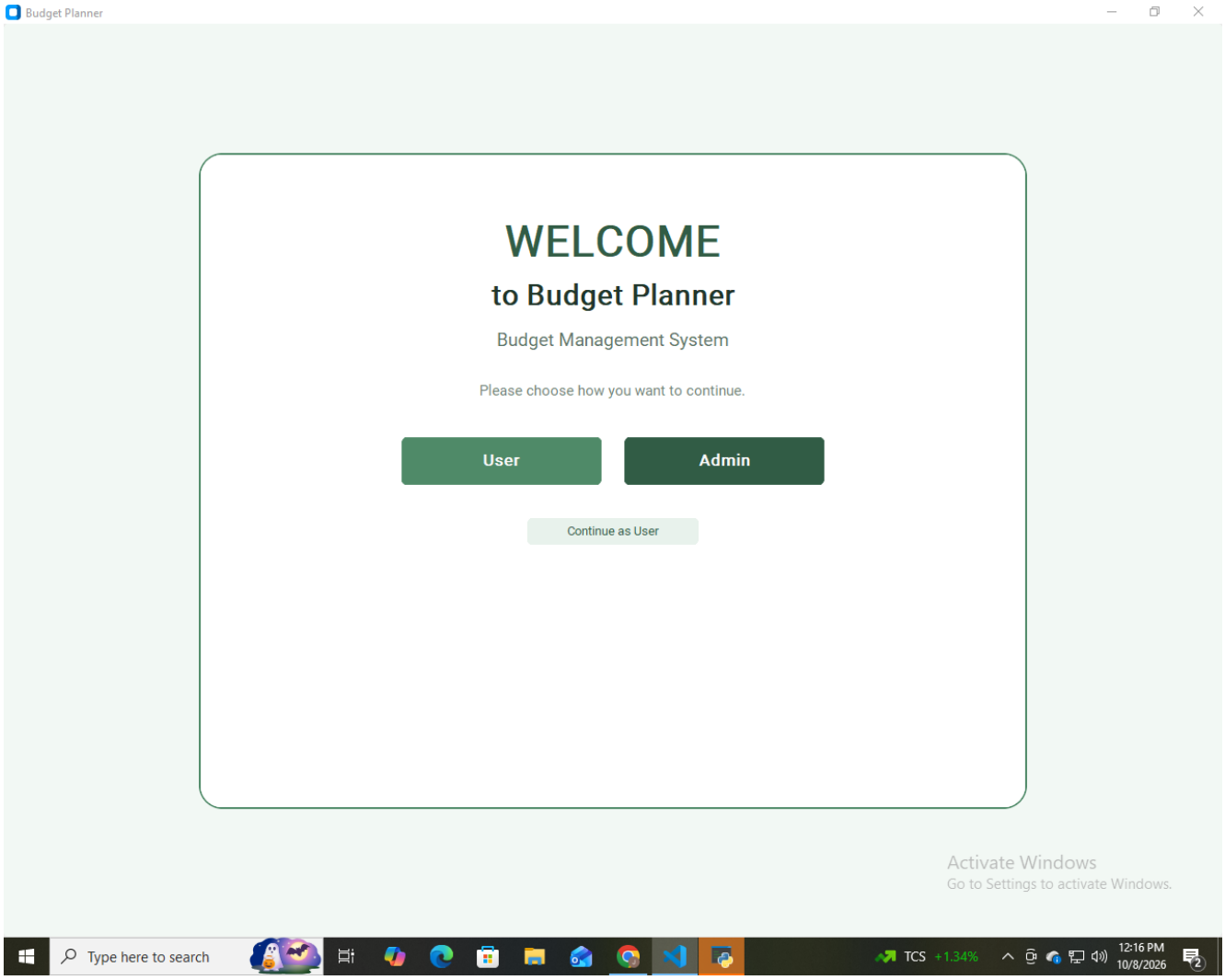


[Database / DB Browser]

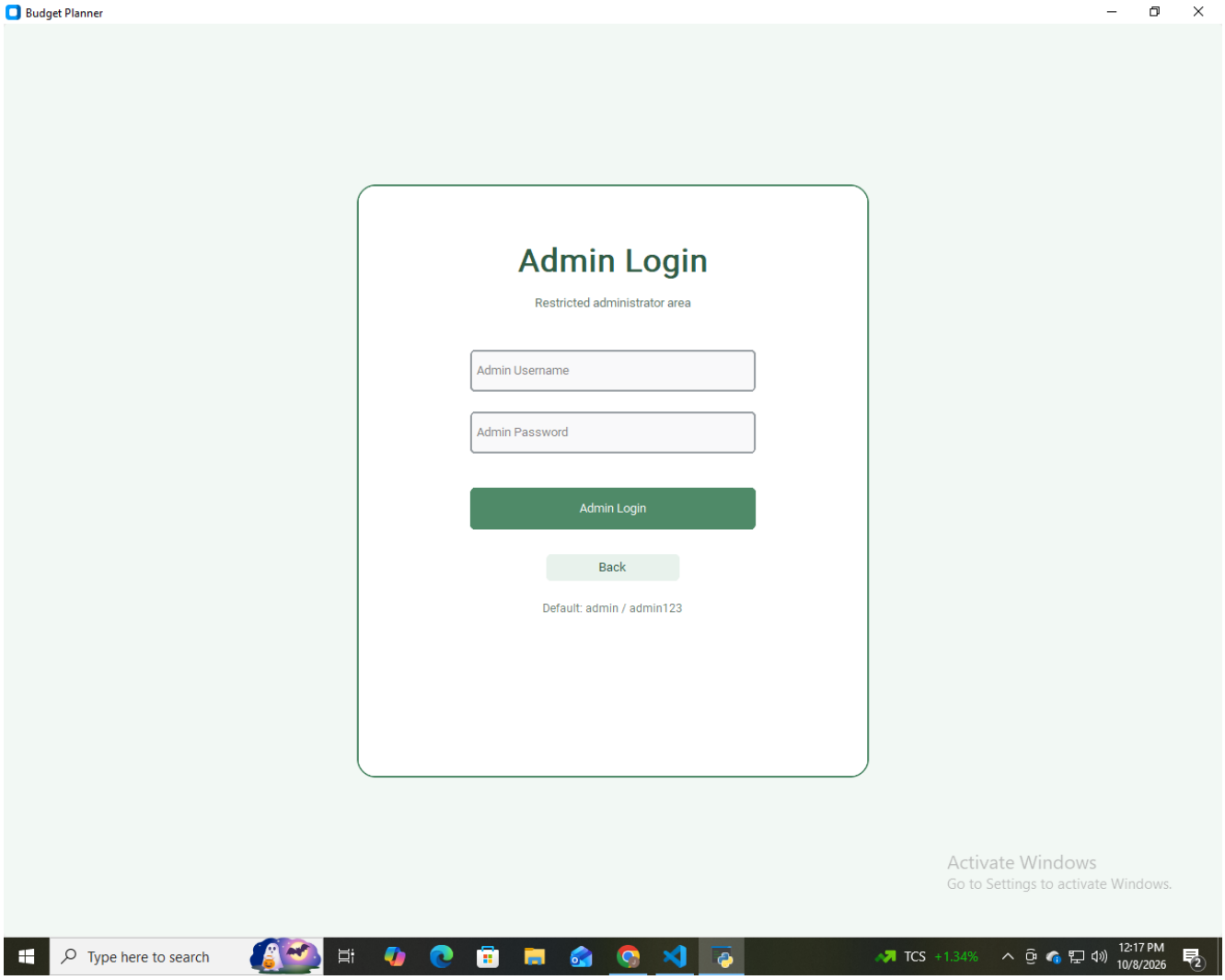
Only screenshots of features that are actually implemented in the final submitted program should be inserted. This prevents the report from claiming features that are not present in the final build.

5.7 Screenshots/ Admin Interface

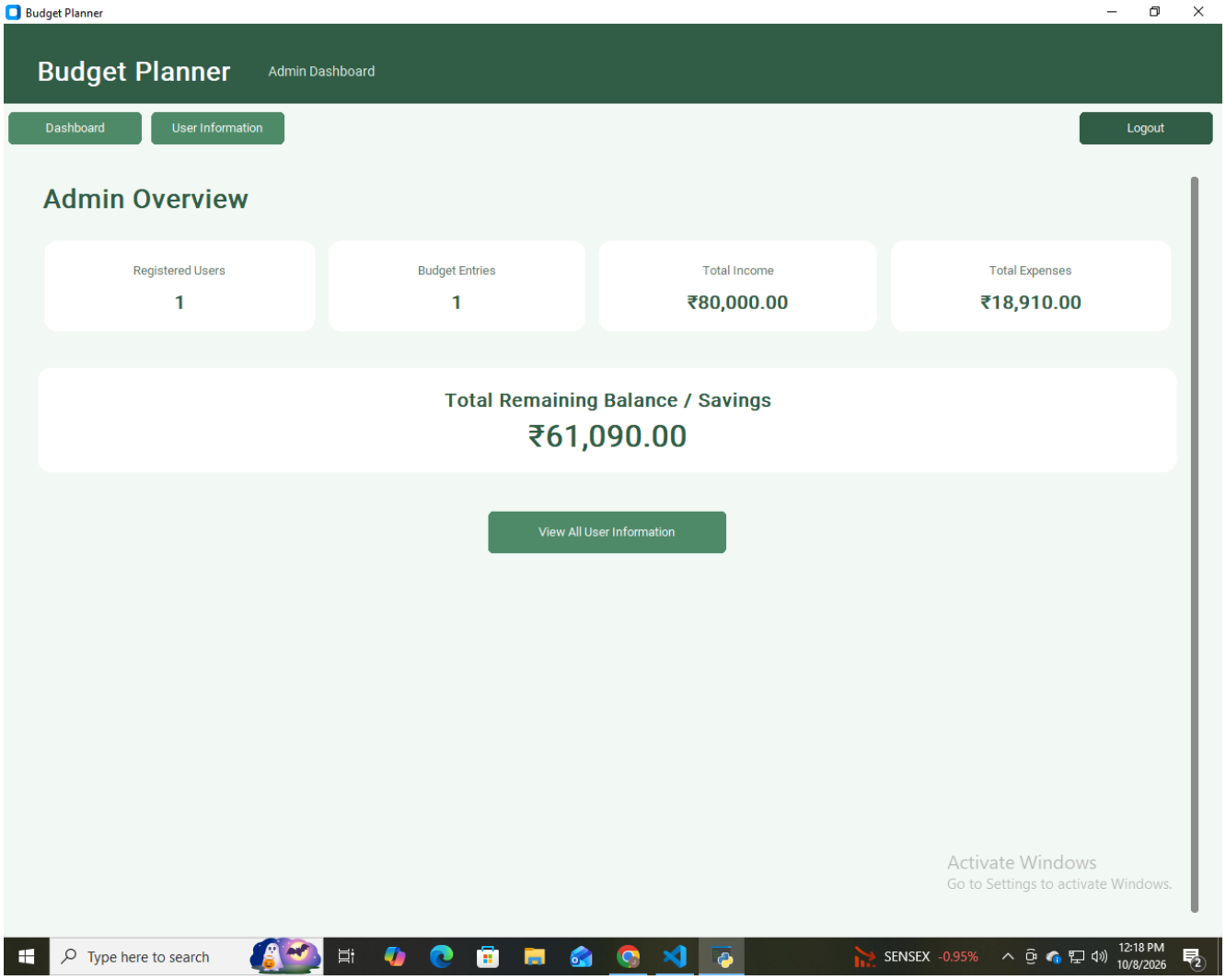
Screenshots are an important part of the project report because they demonstrate the actual interface developed during implementation. The following spaces can be replaced with screenshots from the admin panel. These screenshots are from an admin panel. So, the admin can check the user's information.



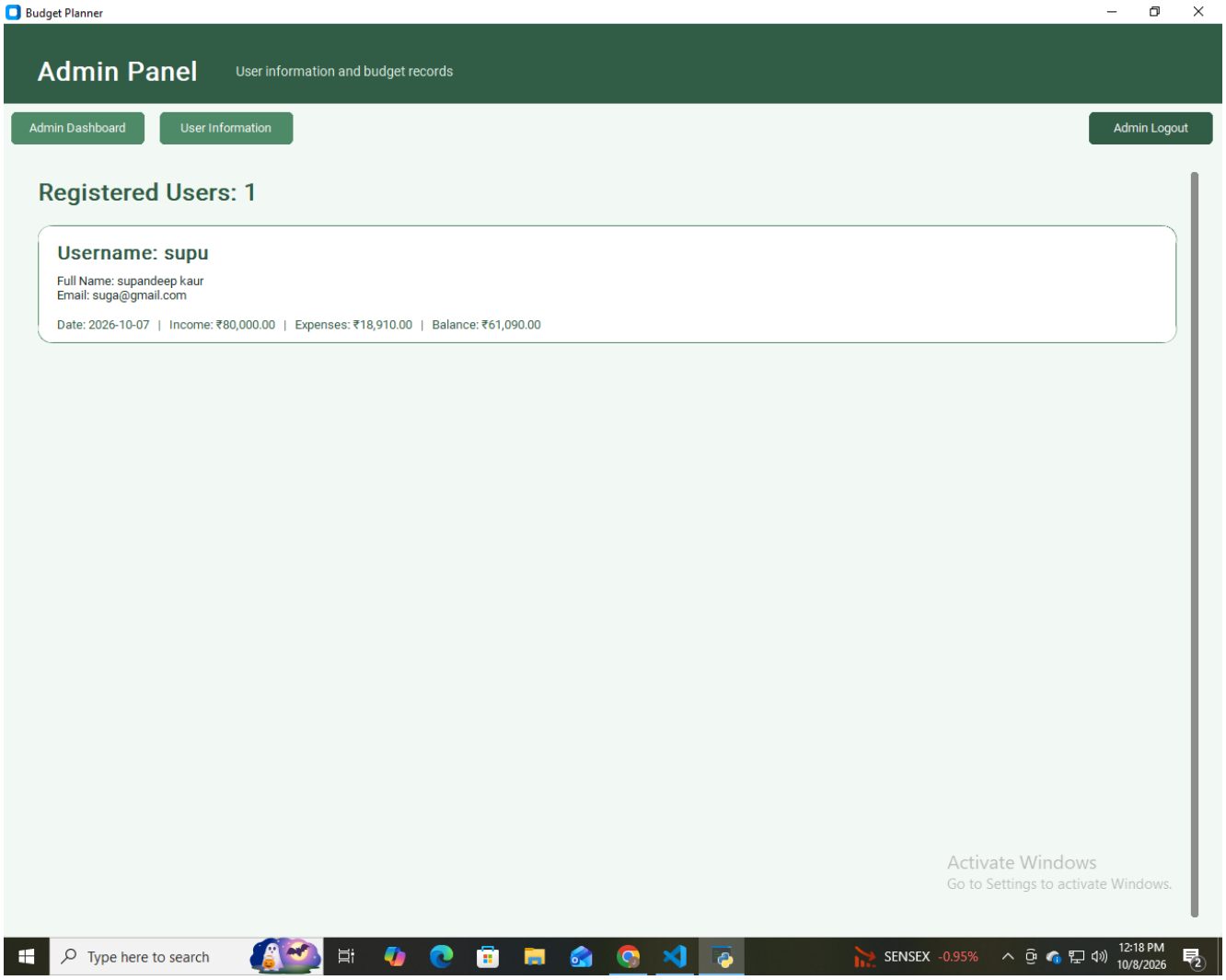
[Welcome Page]


Budget Planner
Admin Login
Restricted administrator area
Admin Username
Admin Password
Admin Login
Back
Default: admin / admin123
Activate Windows
Go to Settings to activate Windows.
Type here to search
TCS +1.34%
12:17 PM
10/8/2026

[Login Page]



[Admin Dashboard]



[User's Information Page]

CHAPTER 6 – PROJECT TESTING / RESULTS

6.1 TEST PLAN

Testing is performed to determine whether the application works according to the requirements. The test plan covers account creation, login, validation, navigation, database operations and budget information.

Feature	Testing Objective
Create Account	Check valid and invalid registration
Password confirmation	Check mismatch handling
Duplicate username	Check database constraint handling
Login	Check valid and invalid credentials
Dashboard input	Check fields accept required information
Navigation	Check buttons open correct pages
Database	Check records are stored/retrieved
Graph	Check expense data is represented correctly

6.2 TEST CASES

Test Case	Input / Condition	Expected Result	Status
TC01	All signup fields valid	Account created	Pass
TC02	Name/email/ username missing	Warning displayed	Pass

TC03	Password and confirmation different	Password error displayed	Pass
TC05	Valid login	Dashboard opens	Pass
TC06	Invalid login	Login error / no access	Pass
TC07	Valid date and income	Data accepted	Pass
TC08	Navigation button clicked	Correct page opens	Pass
TC09	Database insert	Record saved	Pass
TC10	Database read	Stored data displayed	Pass

The status values should be updated if final testing produces different results.

6.3 UNIT TEST AND 6.4 USER TESTING

Unit Testing

Unit testing checks individual functions or small sections of the program. In this project, account creation, validation, navigation and database operations can be tested separately before complete system testing.

Test create_account() with empty fields.
Test create_account() with mismatched passwords.
Test duplicate username handling.
Test successful account insertion.
Test page navigation functions.
Test database connection and closing.

User Testing

User testing checks whether a person can understand and operate the application without detailed knowledge of the source code. The user should be able to identify the login and signup buttons, enter information and move to the dashboard.

6.5 PRESENTATION INTERFACE TESTING

Interface testing checks whether GUI components work correctly and whether messages are understandable.

- Check button labels and navigation.
- Check entry fields and accept appropriate input.
- Check password fields to hide passwords.
- Check warning and error messages appear when required.
- Check the layout remains readable at the intended window size.
- Check database errors do not leave the application in an unexpected state.

6.6 VERIFICATION AND VALIDATION

Verification asks whether the software has been developed according to the design and requirements. Validation asks whether the final application actually satisfies the user's intended need.

6.7 SETUP AND MAINTENANCE

Maintenance may include correcting bugs, improving validation, updating the interface, adding new reports and changing database structures when requirements grow. Before modifying database tables, a backup copy of the database should be kept.

CHAPTER 7 – IMPLEMENTATION EVALUATION

7.1 EFFECTIVENESS OF TESTING

Testing is effective when the main requirements are checked and errors are identified before final submission. The project testing focuses on the functions most important to the user: account creation, login, navigation, budget entry and database storage.

7.2 LIMITATIONS

The project is intended for basic personal budget management rather than professional accounting.
The application depends on the local computer and SQLite database.
Advanced authentication and encryption are outside the current beginner-level scope.
Reporting and export facilities may be limited.
The final interface depends on the exact modules implemented in the submitted source code.

7.3 FUTURE SCOPE

Add stronger password protection and secure password hashing.
Add monthly and yearly reports.
Add export to Excel/PDF.
Add more advanced charts and filters.
Add backup and restore functions.
Add search and sorting of records.
Add multiple-user administration.
Improve responsive layout and accessibility.

CHAPTER 8- CONCLUSION

The Budget Management System demonstrates how Python, CustomTkinter and SQLite can be combined to create a practical desktop application. The system provides a structured way to create an account, log in and manage budget-related information through a graphical interface.

The project helped in understanding GUI programming, database connectivity, SQL operations, input validation, exception handling and software testing. The iterative approach made it possible to develop the application page by page and improve it during testing.

The project can be extended in the future with more secure authentication, advanced reporting, data export and additional analytical features.

BIBLIOGRAPHY / REFERENCES

Python Software Foundation, Python Documentation. https://docs.python.org/3/

Python Software Foundation .sqlite3 —DB-API 2.0 interface for SQLite databases. https://docs.python.org/3/library/sqlite3.html

SQLite consortium. SQLite Documentation. https://www.sqlite.org/docs.html

CustomTkinter project Documentation. https://customtkinter.tomschimansky.com/

TkDocs.Tkinter documentation and tutorials. https://tkdocs.com/

Visual Studio Code Documentation. https://code.visualstudio.com/docs